Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 02

Desarrollo de aplicaciones Web: Inversión de Control y Gestión de Dependencias

DOCENTE:

Coello Palomino, Ricardo

CURSO:

Desarrollo de aplicaciones Web
4 - C24 - Sección A - B

DESARROLLO DE APLICACIONES WEB AVANZADO: INVERSIÓN DE CONTROL Y GESTIÓN DE DEPENDENCIAS

I. Capacidades

Implementar y comprender cómo Spring Boot gestiona objetos mediante IoC y DI, utilizando anotaciones y configuraciones prácticas.

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

III. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

IV. Normas empleadas

- *No aplica*

V. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 10.
- La instalación del software requerido se realizará en el equipo virtual.

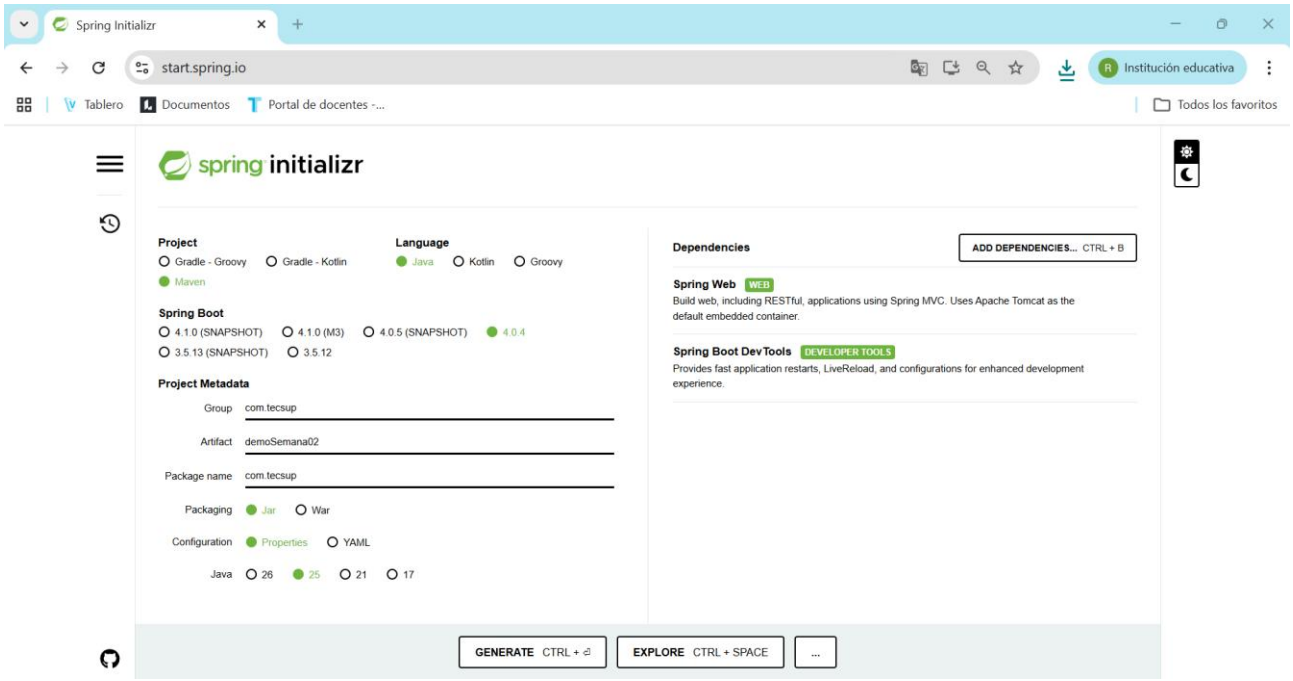
VI. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual.

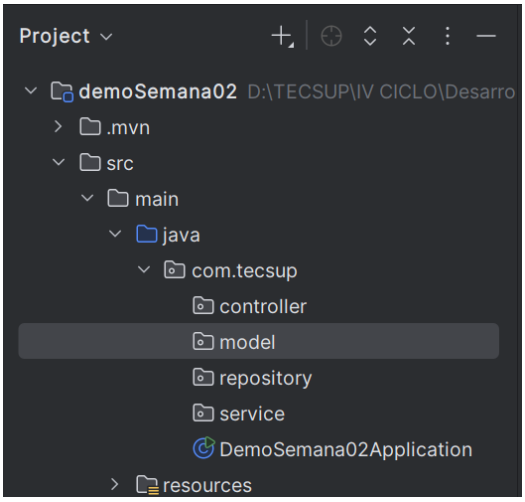
VII. Procedimiento

Laboratorio 01

1.- Crear proyecto en Spring Initializr:



2.- Estructura:



3.- Actividad 1: Inyección básica

Paso 1: Crear modelo (Entidad simple)

```java
package com.tecsup.model;

public class Usuario {
    private String nombre;
    private String email;

    public Usuario(String nombre, String email) {
        this.nombre = nombre;
        this.email = email;
    }
```

```
    public String getNombre() {
        return nombre;
    }

    public String getEmail() {
        return email;
    }
}
```

Paso 2: Crear servicio (lógica de negocio)

```
package com.tecsup.service;

import com.tecsup.model.Usuario;
import org.springframework.stereotype.Service;

@Service
public class UsuarioService {

    public Usuario obtenerUsuario() {
        // Simulamos datos (como si vinieran de BD)
        return new Usuario("Ricardo", "ricardo@email.com");
    }
}
```

Aquí ocurre IoC:
Spring crea automáticamente UsuarioService (no usamos new)

Paso 3: Crear controlador (inyección de dependencia)

```
package com.tecsup.controller;

import com.tecsup.model.Usuario;
import com.tecsup.service.UsuarioService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;


@RestController
public class UsuarioController {

    @Autowired
    private UsuarioService usuarioService;

    @GetMapping("/usuario")
    public Usuario obtenerUsuario() {
        return usuarioService.obtenerUsuario();
    }
}
```

Aquí ocurre DI:
Spring inyecta automáticamente UsuarioService dentro del controlador

Paso 4: Resultado esperado (Ejecutar la aplicación)

Abrir en navegador: http://localhost:8080/usuario



API DE USUARIOS CON MYSQL

BACKEND

1. Base de datos MySQL

```
CREATE DATABASE demo_spring;

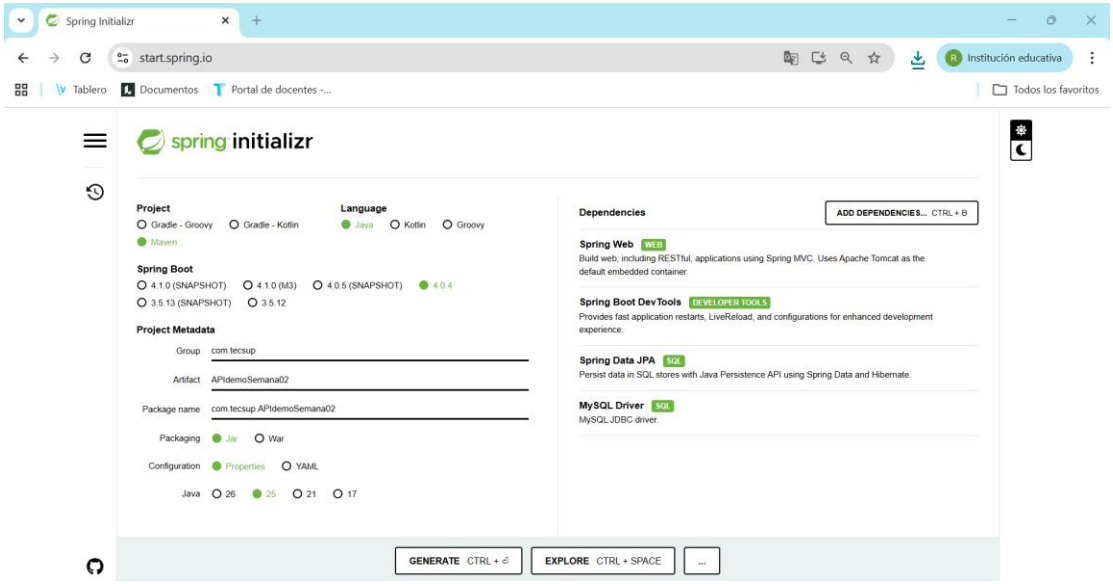
USE demo_spring;

CREATE TABLE usuario (
   id INT AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(100),
   email VARCHAR(100)
);

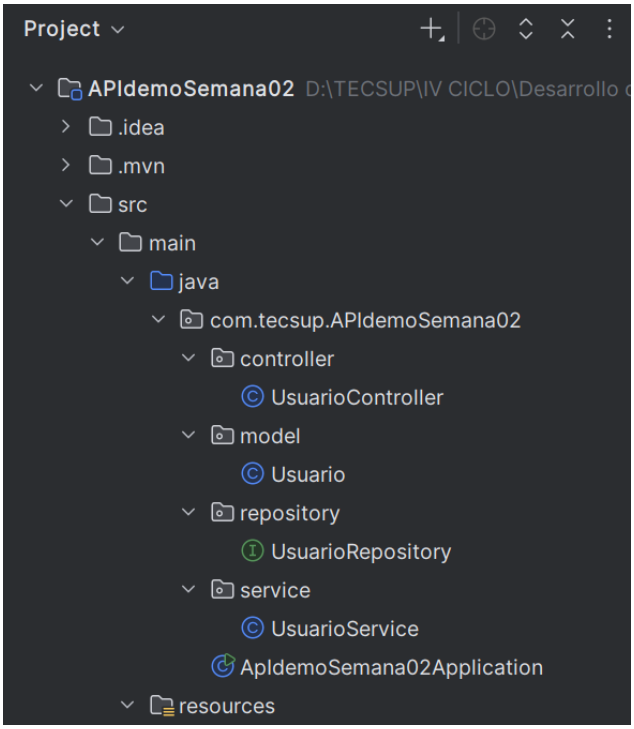
INSERT INTO usuario(nombre, email) VALUES
('Ricardo', 'ricardo@email.com'),
('Ana', 'ana@email.com');

SELECT * FROM usuario;
```

2.- Configuración initializr



3. Crear la siguiente estructura de carpetas y archivos.



4.- Configuración de la base de datos en el archivo "**application.properties**"

```
spring.datasource.url=jdbc:mysql://localhost:3306/demo_spring?useSSL=false&serve
rTimezone=UTC
spring.datasource.username=root
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
```

5.- Crear entidad Usuario

```
package com.tecsup.APIdemoSemana02.model;

import jakarta.persistence.*;

@Entity
@Table(name = "usuario")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String nombre;
    private String email;

    // Constructor vacío (OBLIGATORIO)
    public Usuario() {}

    public Usuario(String nombre, String email) {
        this.nombre = nombre;
        this.email = email;
```

```
    }

    // Getters y Setters

    public int getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    public String getEmail() {
        return email;
    }
}
```

6.- Crear UsuarioRepository

```
package com.tecsup.APIdemoSemana02.repository;

import com.tecsup.APIdemoSemana02.model.Usuario;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UsuarioRepository extends JpaRepository<Usuario, Integer> {
}
```

7.- Crear UsuarioService

```
package com.tecsup.APIdemoSemana02.service;

import com.tecsup.APIdemoSemana02.model.Usuario;
import com.tecsup.APIdemoSemana02.repository.UsuarioRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class UsuarioService {

    @Autowired
    private UsuarioRepository usuarioRepository;

    public List<Usuario> listarUsuarios() {
        return usuarioRepository.findAll();
    }
}
```

8.- Crear UsuarioController

```
package com.tecsup.APIdemoSemana02.controller;

import com.tecsup.APIdemoSemana02.model.Usuario;
import com.tecsup.APIdemoSemana02.service.UsuarioService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
```

```
@RestController
@RequestMapping("/usuarios")
public class UsuarioController {

    @Autowired
    private UsuarioService usuarioService;

    @GetMapping
    public List<Usuario> listarUsuarios() {
        return usuarioService.listarUsuarios();
    }
}
```

9.- Ejecutar la aplicación



FRONTEND

10.- Agregar dependencia (pom.xml)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

11.- Crear HTML en templates

Ruta:

src/main/resources/templates/usuarios.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Usuarios</title>

    <!-- Bootstrap -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
```

```
</head>

<body class="bg-light">

<div class="container mt-5">

    <h2 class="text-center mb-4">Lista de Usuarios</h2>

    <div class="card shadow">
        <div class="card-body">

            <table class="table table-striped table-hover table-bordered">
                <thead class="table-dark">
                <tr>
                    <th>ID</th>
                    <th>Nombre</th>
                    <th>Email</th>
                </tr>
                </thead>

                <tbody>
                <tr th:each="u : ${usuarios}">
                    <td th:text="${u.id}"></td>
                    <td th:text="${u.nombre}"></td>
                    <td th:text="${u.email}"></td>
                </tr>
                </tbody>

            </table>

        </div>
    </div>

</div>

</body>
</html>
```

12.- Modificar el Controller

```
package com.tecsup.APIdemoSemana02.controller;

import com.tecsup.APIdemoSemana02.service.UsuarioService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/usuarios")
public class UsuarioController {

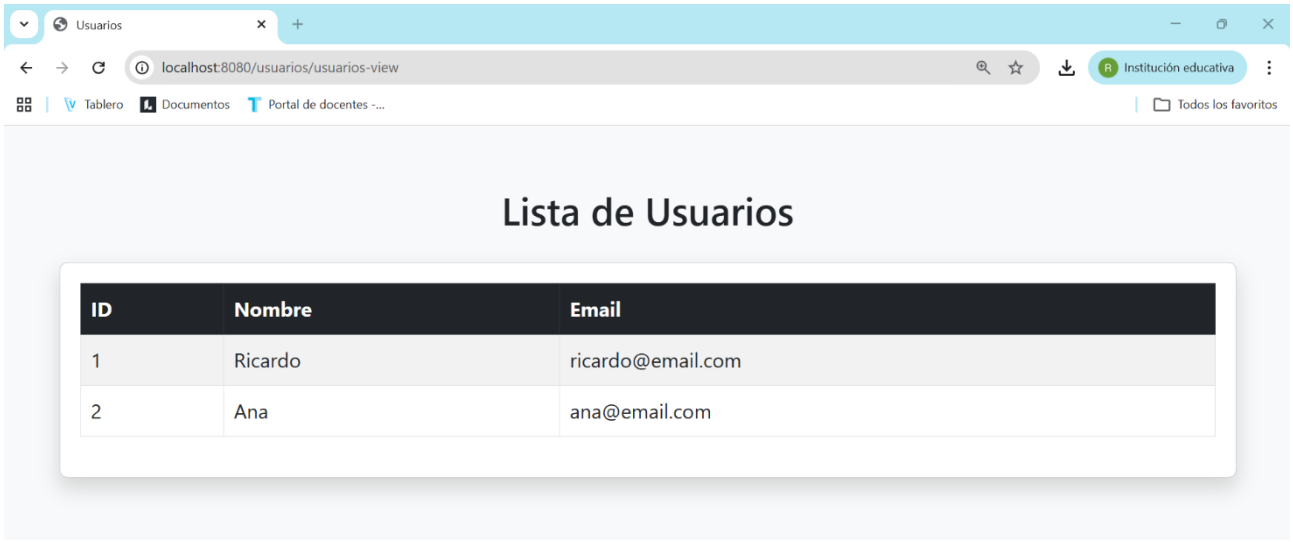
    private final UsuarioService usuarioService;

    public UsuarioController(UsuarioService usuarioService) {
        this.usuarioService = usuarioService;
    }

    @GetMapping("/usuarios-view")
    public String verUsuarios(Model model) {
        model.addAttribute("usuarios", usuarioService.listarUsuarios());
        return "usuarios"; // nombre del HTML
```

```
        }
}
```

13.- Ejecutar la aplicación



LABORATORIO: Sistema Web de Productos

Objetivo

Desarrollar una aplicación web utilizando **Spring Boot** que permita visualizar información almacenada en una base de datos MySQL, aplicando el patrón de arquitectura por capas (Controller, Service, Repository) y mostrando los datos en una vista HTML con estilo Bootstrap.

Descripción de la actividad

La empresa **TechStore** necesita un sistema web básico para visualizar su lista de productos. Como desarrollador, deberás construir una aplicación que permita mostrar esta información desde la base de datos en una tabla web.

Requerimientos

- **Base de Datos**

  - Crear una base de datos en MySQL.
  - Crear una tabla llamada **producto** con los siguientes campos:
    - id
    - nombre
    - precio
  - Insertar al menos **3 registros**.

- **Backend (Spring Boot)**

Implementar las siguientes capas:

- **Entidad (Entity)**
  Representar la tabla producto.

- **Repository**
  Permitir la conexión con la base de datos.
- **Service**
  Gestionar la lógica de negocio.
- **Controller**
  Crear una ruta que permita mostrar los productos en una vista web.

🔹 **Vista (Frontend)**

- Crear una vista HTML usando **Thymeleaf**.
- Mostrar los productos en una **tabla**.
- Aplicar estilos utilizando **Bootstrap**.

Resultado esperado

Al ejecutar la aplicación y acceder a la URL correspondiente, se debe visualizar:

- Una tabla con los productos registrados
- Datos cargados desde MySQL
- Interfaz mejorada con Bootstrap

DESARROLLO DE LABORATORIO:

BASE DE DATOS:

```
Query: CREATE TABLE producto ( id INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(100), precio DECIMAL(10, 2) )

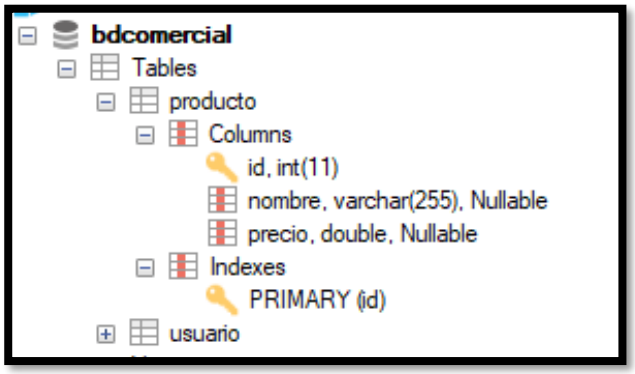
0 row(s) affected

Execution Time : 0.027 sec
Transfer Time  : 1.014 sec
Total Time     : 1.041 sec
-------------------------------------------------------------

Query: INSERT INTO producto (nombre, precio) VALUES ('Laptop HP Victus 15', 3500.50), ('Xiaomi 15', 3150.00),
('Audifonos Redmi', 180.0...

3 row(s) affected

Execution Time : 0.007 sec
Transfer Time  : 0 sec
Total Time     : 0.008 sec
```




```
complete: [Tab]->Next Tag. [Ctrl+Space]->List All Tags. [C
  USE dbcomercial;

CREATE TABLE producto (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100),
    precio DECIMAL(10, 2)
);

  INSERT INTO producto (nombre, precio) VALUES
  ('Laptop HP Victus 15', 3500.50),
  ('Xiaomi 15', 3150.00),
  ('Audifonos Redmi', 180.00);
```

BACKEND:

PRODUCTO CONTROLLER:

```java
package com.tecsup.Controller;

import com.tecsup.Services.ProductoService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
@RequestMapping("/productos")
public class ProductoController {

    private final ProductoService productoService;  2 usages

    public ProductoController(ProductoService productoService) {
        this.productoService = productoService;
    }

    @GetMapping("/vista")
    public String verProductos(Model model) {
        model.addAttribute( attributeName: "productos", productoService.listarProductos());
        return "productos";
    }
}
```

PRODUCTO:

```java
package com.tecsup.Model;

import jakarta.persistence.Entity;
import jakarta.persistence.Table;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;

@Entity
@Table(name = "producto")
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String nombre;  2 usages

    private Double precio;  2 usages

    public Producto() {}

    public Producto(String nombre, Double precio) {  no usages
        this.nombre = nombre;
        this.precio = precio;
    }

    public int getId() { return id; }  no usages
    public String getNombre() { return nombre; }  no usages
    public Double getPrecio() { return precio; }  no usages
}
```

PRODUCTO REPOSITORY:

```java
package com.tecsup.Repository;

import ...

public interface ProductoRepository extends JpaRepository<Producto, Integer> {  2 usages
}
```

PRODUCTO SERVICE

```java
package com.tecsup.Services;

import com.tecsup.Model.Producto;
import com.tecsup.Repository.ProductoRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service  3 usages
public class ProductoService {

    @Autowired
    private ProductoRepository productoRepository;

    public List<Producto> listarProductos() {  1 usage
        return productoRepository.findAll();
    }
}

```

PRODUCTOS.HTML:

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>TechStore</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light">
<div class="container mt-5">
    <h2 class="text-center mb-4">Lista de Productos</h2>
    <div class="card shadow">
        <div class="card-body">
            <table class="table table-striped table-hover table-bordered">
                <thead class="table-dark">
                <tr>
                    <th>ID</th>
                    <th>Nombre</th>
                    <th>Precio</th>
                </tr>
                </thead>
                <tbody>
                <tr th:each="p : ${productos}">
                    <td th:text="${p.id}"></td>
                    <td th:text="${p.nombre}"></td>
                    <td th:text="${p.precio}"></td>
                </tr>
                </tbody>
            </table>
        </div>
    </div>
</div>
</body>
</html>
```

RESULTADO FINAL:



Conclusiones:

Indicar 5 conclusiones que llegó después de los temas tratados de manera práctica en este laboratorio.

- Pude comprobar en la práctica cómo Spring Boot te facilita la vida con la Inyección de Dependencias. En lugar de estar instanciando objetos a cada rato de forma manual, usar anotaciones como @Autowired o inyectar por constructor hace el trabajo pesado por detrás y deja el código mucho más limpio.

- Trabajar con la arquitectura por capas (Model, Repository, Service, Controller) al principio parece dar más vueltas, pero ya armando el proyecto me di cuenta de que te ayuda un montón a mantener el orden. Separar la conexión a la base de datos de la lógica de negocio y de las vistas hace que sea más fácil encontrar errores.

- Me pareció súper útil cómo Thymeleaf conecta el backend de Java directo con el HTML. Fue genial ver cómo la lista de productos que jalaba de MySQL pasaba directo a una tabla de Bootstrap usando solo unos cuantos atributos como th:each y th:text en la vista.

- Aprendí a la mala que configurar bien el archivo application.properties y sincronizar las dependencias en Gradle es vital. Me di cuenta de que si el nombre de la base de datos falla por una sola letra, o si olvidas recargar el elefantito de Gradle tras agregar una librería, todo el servidor colapsa o no reconoce las páginas.

- El uso de Spring Data JPA simplifica muchísimo la conexión a la base de datos. Me sorprendió que con solo extender la interfaz JpaRepository ya tenía acceso a los métodos para listar los registros de mi tabla en MySQL, sin tener que escribir consultas SQL complejas desde cero.